

Connor

AI 前沿情报 Daily Agent 技术文档

面向面试的项目技术说明 · Agent 系统设计叙事
仓库: <https://github.com/caojiajun777/Connor>

一句话定位: 一个可恢复、可审计的多阶段 Agent 系统, 从 X (Twitter) 增量采集 AI 前沿信号, 经工具调用、绝对评分与主编级精选, 自动生产并发布中文 AI 早报。

字段	内容
文档用途	面试项目讲解 / 系统设计问答备课
系统形态	Staged Multi-Agent Pipeline (非自由 ReAct)
核心产出	每日 Top≤20 事件早报 + 公开站 + 内部 Console
版本锚点	Daily Agent Spec v1 · Watchlist v2 · 2026

1. 项目用了什么（技术栈）

Connor 是全栈 Agent 工程，而不是单次 Prompt Demo。技术选型按「感知层 / 决策层 / 记忆层 / 产品层」拆分。

1.1 分层技术栈

层次	技术	作用
感知 / Tool	TypeScript · Playwright · MCP · 持久 Chrome 实例	网页抓取、搜索/主页/单帖；会话与凭证隔离在本地 profile
编排 / Runner	Python 3.11+ · Pydantic · PyYAML · CLI	Watchlist 增量采集、清洗、覆盖率与游标推进
Agent Runtime	LangGraph · PostgreSQL Checkpointer · 图数据库	固定阶段图、可恢复 run、advisory lock、工作游标
决策 / LLM	DeepSeek API（可配置）· 版本化 Prompt	摘要、四维绝对评分、主编精选、事件打包、写稿
存储	PostgreSQL · Redis · 本地/S3 媒体	帖子/评分/入选/日报持久化；游标与 outbox
产品	FastAPI · React/Vite Console · Next.js	公开/内部标注与运维；对外早报阅读与归档
运维	Windows 计划任务 · PowerShell · Cloudflare	定时日报、内容暴露、媒体同步

1.2 为什么这样选（面试可用理由）

- MCP + Playwright：在没有稳定官方 API 预算时，把浏览器能力封装成 Agent 可调用 Tool，边界清晰。
- LangGraph 固定阶段：情报流水线需要可审计、可恢复，不适合完全自由 ReAct（易漏采、难回放）。
- PG + Redis：真相源在 PG；Redis 做无 TTL 工作游标，兼顾吞吐与可重建。
- 绝对评分再主编精选：先 pointwise 可比，再 listwise 做编辑判断，平衡成本与质量。
- 入选 ≠ 发布：把决策与上线解耦，符合内容产品的人工/自动边界。

2. 怎么做的（Agent 架构与流水线）

对外包装成「Agent 项目」时，建议强调：这是一个带 Tool、Memory、Policy、Gate 的生产级 Agent，不是聊天机器人套壳。

2.1 Agent 角色拆分

Agent / 模块	职责	输入 → 输出
Collector Agent	调用 MCP Tool 增量抓取	Watchlist + Cursor → 原始帖子
Cleaner	结构化清洗与字段补全	raw → x-clean-posts
Summarizer	忠实中译 / 内容类型标注	原文 → summary + content_type
Evaluator	单帖绝对四维打分	原文 → 4 scores + category
Ranker（程序）	字典序 Top K	全量评分 → Top K=50
Editor Agent	全局精选 Top N	Top K 卡片 → Top N≤20
Packager + Writer	事件包与早报正文	入选帖 → 事件 + 中文 digests
Publisher	媒体同步与公开上线	草稿报告 → 已发布日报

2.2 主流水线 (Daily Agent Spec)

DailyTrigger

```
→ acquire_run_lock → initialize_run■■■■ watchlist/prompt/top_k ■■■■
→ collect_accounts_loop■MCP ■■■■■■ 72h ■■■■
→ cursor_sync_gate / collection_gate
→ freeze_candidate_snapshot
→ summarize_all → summary_gate
→ evaluate_all → evaluation_gate
→ select_top_k■■■■■■■■■■
→ editorial_final_selection■LLM Top≤20■
→ persist_selection → write_report → publish
→ finalize_run → release_run_lock
```

关键产品原则：游标只定义增量边界，不限制条数；候选不能被静默丢弃；run 必须可恢复；所有候选都要有被评估的机会。

2.3 Tool 层: X News MCP

- 工具: x_session_status / x_search_posts / x_profile_posts / x_get_post
- 只读策略: 不点赞、不关注、不发帖; 专用 Chrome profile, 凭证不进仓库
- 采集策略: 串行账号、会话复用、页数安全上限、精确命中旧游标即停
- cursor_eligible: 裸转发/置顶不可作游标锚点, 避免边界错乱

2.4 决策层核心算法

(1) Pointwise 绝对评分

每条帖独立打分, 不与同批次互比。四维: importance / information_gain / specificity / frontier (0-10)。打分读原文, 中译仅作辅助。前沿未证实爆料在 Prompt 中被定义为高价值信号 (通常 ≥ 8), 不以「缺官宣」降权。

(2) 字典序 Top K (非正则、非加权总分)

排序键为元组降序比较: importance → information_gain → specificity → frontier → published_at → post_id。前一维略高即可压过后三维; 可复现、可单测。默认 Top K=50。

(3) Listwise 主编精选 Top N

将 Top K 卡片一次交给 Editor LLM, 做去重、多角度保留、爆料优先与叙事平衡。代码硬约束: post_id 必须属于 Top K、去重、截断到 Top N=20、rank 重编号。「爆料必须入选」目前是 Prompt 策略约束, 可在面试中主动谈 hardening 空间 (程序回填)。

2.5 Memory / 可靠性

- PostgreSQL: run / posts / summaries / evaluations / selections / reports 真相源
- Redis: connor:x:cursor:<handle> 工作游标 (无 TTL) + outbox 同步
- LangGraph Checkpointer: 生产 start/resume, 配合 PG advisory lock 防并发双跑
- Gate: summary_gate / evaluation_gate / collection_gate; 支持 accept_partial 显式降级
- 版本冻结: prompt hash、top_k/top_n、模型版本随 run 固化, 保证可回放

3. 做成了什么（效果与交付物）

3.1 产品交付

- 增量情报采集系统: Watchlist 覆盖官方 / 员工 / 分析师 / 爆料源 (约 120+ 账号)
- Daily Agent: 定时跑通采集→评分→精选→写稿→发布
- 公开站 (Next.js): 按日阅读 AI 早报、归档、媒体画廊
- 内部 Console: run/评分/标注、Watchlist 管理与账号审计
- 只读 FastAPI: /runs /selection /evaluations /api/public/* /media/*

3.2 工程效果（可量化叙事）

维度	效果
候选公平性	游标间新帖全量进候选，不做「每号只留 10 条」式业务截断
决策结构	绝对分粗筛 (K=50) + 主编精选 (N=20) ， 兼顾成本与编辑质量
可恢复性	Gate + checkpoint + resume； 缺口需显式 accept， 禁止静默丢数据
内容质量策略	提高 leak 源日配额； 剔除低信号聚合号； 前沿爆料优先入选
产品闭环	入选与发布解耦； 公开站只读已发布报告
可观测	coverage 区分 fetch_empty / empty_window； session reason_code 可诊断登录态

3.3 典型输出形态

每日产出「AI 早报 YYYY-MM-DD」：导语 + 多事件条目（标题/导读/正文/外链）+ 关键词；事件默认一帖一事，允许官方重复通告合并与同源评测卡片合并。公开页强调证据链与媒体，而不是营销文案。

4. 系统架构图（面试口述版）



口述要点: Tool 解决「怎么看见世界」; Gate/Cursor 解决「怎么可靠」; Evaluator/Editor 解决「怎么判断」; Publisher 解决「怎么交付」。

5. 面试问答弹药（建议背这几条）

Q1：为什么说这是 Agent，而不是爬虫 + LLM？

A：有 Tool 抽象（MCP）、有状态记忆（cursor/PG/checkpoint）、有策略与门禁（eligibility/gate）、有多角色协作（采集/评分/主编/写稿），并且 run 可恢复。爬虫只管抓；Agent 要在约束下完成目标闭环。

Q2：为什么不用端到端一个大 Prompt 直接出早报？

A：不可审计、不可恢复、成本高、难控漏采。我们把「感知」与「决策」拆开：先保证候选完整，再绝对评分，再小范围 listwise 精选。

Q3：四维评分为什么用字典序而不是加权和？

A：加权和会把不同语义维度混成一个标量，权重难解释且易被调参游戏绑架。字典序把 importance 设为硬优先级，保证「先重要，再信息增益」的产品语义清晰、可复现。

Q4：未证实爆料为何还要高分入选？

A：产品目标是前沿情报，不是新闻社核实台。未证实但具体可归因的信号（model ID、灰度、蒸馏争议）是最高价值输入；写稿阶段保留不确定性与归因，而不是在筛选阶段误杀。

Q5：最大技术难点是什么？

A：增量边界正确性（cursor_eligible / known_data_gap）、浏览器会话稳定性、以及「程序排序确定性」与「LLM 编辑主观性」之间的契约。我们用 Gate、版本冻结和 post_id 白名单约束 LLM。

Q6：如果重做，你会改什么？

A：给 frontier_leak 做程序级 must-include；事件打包支持清单型爆料拆分；必要时引入官方 API 降低浏览器脆弱性；把 Editor 输出做成可解释的约束求解而不仅是 Prompt。

Q7：你在项目中的贡献如何讲？

A：按模块讲：Watchlist 情报源治理；Daily Agent 流水线与评分/精选机制；公开站与发布闭环；可靠性（调度、gate、cursor）。用「问题→设计→验证」三段式，避免只罗列技术名词。

6. 代码地图（方便现场翻）

路径	内容
src/	X News MCP Server (Playwright)
app/x_watchlist/	Watchlist 采集编排 / MCP client / cleaner
app/daily/	Daily Agent: 采集、评分、精选、写稿、发布、API
app/daily/prompts/	版本化 Prompt (summary/eval/editorial/writer)
app/daily/report_writing/	Packager / Writer / Assemble
app/daily/public/	公开 API、媒体同步、发布
config/x_watchlist.yaml	情报源清单 (official/employee/analyst/leak)
docs/agent-design.md	Daily Agent 冻结规格
frontend/	内部 Console (Vite/React)
web/	公开站 (Next.js)
tests/	采集、评分、公开报告等单测

7. 一分钟电梯稿（收尾）

Connor 是一个 AI 前沿情报 Daily Agent。它用 MCP 浏览器工具从 X 增量采集 120+ 专业信源，在 PostgreSQL/Redis/LangGraph 上跑可恢复流水线，先对每条信号做四维绝对评分并用字典序取 Top 50，再由主编 Agent 精选 Top 20，打包写成中文早报并发布到公开站。项目的核心价值不是「会调用 LLM」，而是把 Agent 做成可审计、可恢复、可交付的生产系统。

— 文档结束 —

说明：本文按现有仓库实现整理，用于面试表达；具体数字（账号数/TopK/TopN）以运行配置为准。